FUNDAMENTOS DE INGENIERÍA DE DATOS · TEMAS 2.2 Y 2.5

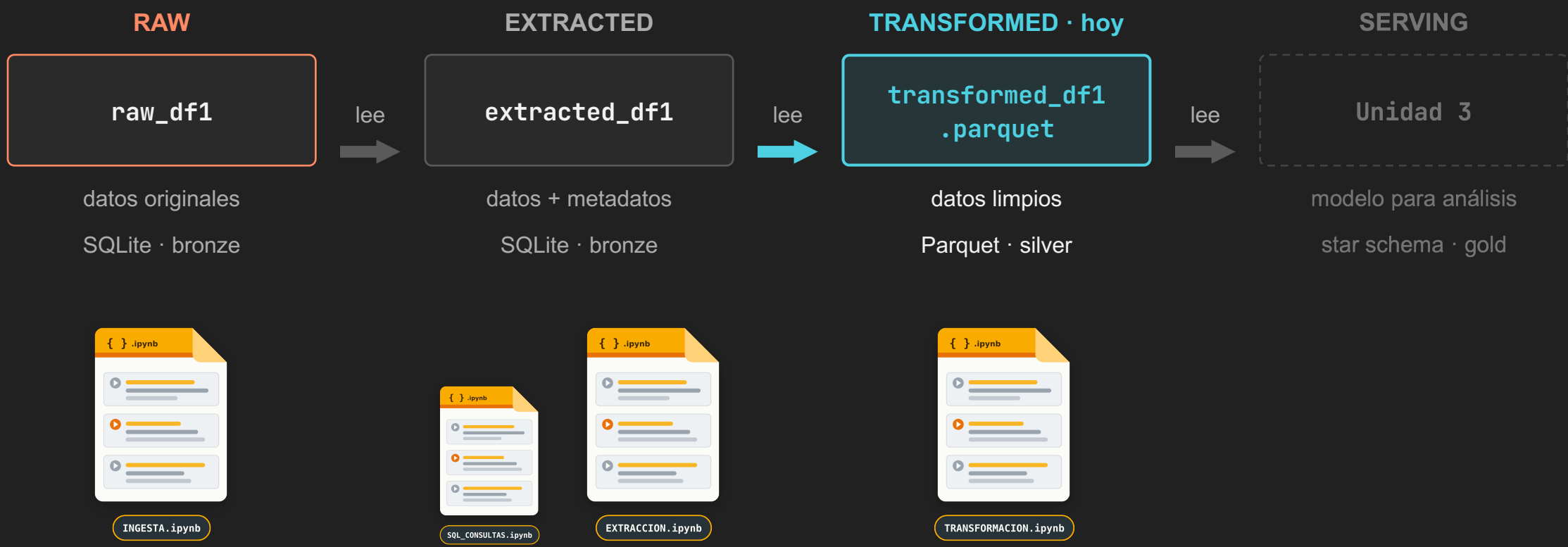
La capa TRANSFORMED

Transformar, cargar a Parquet y validar, paso a paso

TRANSFORMED guarda los datos de EXTRACTED limpios, con tipos correctos y listos para analizar.

Es la primera capa en la que sí cambiamos los datos.

DIAGRAMA · ¿EN QUÉ PARTE DEL PIPELINE ESTAMOS?



Regla de oro: cada capa lee solo de la anterior. Este notebook lee extracted_df1 en modo solo lectura, nunca raw_df1.

Los archivos que genera esta capa

.parquet · .jsonl · .json · .csv

DIAGRAMA · LO QUE QUEDA EN TU DRIVE AL TERMINAR

```
Transformacion_de_Datos/
├── Processed/
│   └── transformed_df1.parquet
├── Logs/
│   ├── transformacion_log.jsonl
│   └── lineage_transformed_df1.json
└── Reportes/
    ├── bitacora_transformed_df1.csv
    └── calidad_transformed_df1.csv
```

Processed · los datos

La capa TRANSFORMED: 16,323 filas limpias en Parquet. Es lo que consume la Unidad 3.

Logs · la historia del pipeline

Qué operaciones se hicieron (log) y de dónde vienen los datos (lineage).

Reportes · las evidencias

Las tablas del reporte de validación del Hito 2, listas para abrir en Excel.

DIAGRAMA · CINCO ARCHIVOS, CUATRO FORMATOS, UN TRABAJO CADA UNO

.parquet	.jsonl	.json	.csv ×2
Qué es	Qué es	Qué es	Qué es
Binario, guardado por columnas	Texto: un JSON por línea	Texto: un solo documento	Texto: una tabla
Qué guarda	Qué guarda	Qué guarda	Qué guarda
Los datos limpios	El log de operaciones	El lineage de la tabla	Bitácora y calidad
Cómo se escribe	Cómo se escribe	Cómo se escribe	Cómo se escribe
Se sobrescribe	Se agrega al final	Se sobrescribe	Se sobrescribe
Quién lo lee	Quién lo lee	Quién lo lee	Quién lo lee
pandas, DuckDB, Unidad 3	Quien opera y monitorea	Quien audita el pipeline	Excel y tu reporte

ANATOMÍA · ARCHIVO JSON LINES (TRANSFORMACION_LOG.JSONL)

```
{"timestamp": "2026-10-28T…",
 "etapa": "carga_parquet",

 "origen": "extracted_df1",
 "destino": ".../Processed/transformed_df1.parquet",

 "filas_entrada": 16598, "filas_salida": 16323,

 "version": "v1", "estado": "exitoso",
 "detalle": "0.75 MB"}

{"timestamp": "2026-10-28T…", "etapa": "carga_parquet", …}
```

Una línea = una operación
Cuándo ocurrió y qué etapa fue.

De dónde a dónde
El recorrido de esta operación.

Cuántas filas entraron y salieron
Números sin comillas.

Con qué reglas y cómo terminó
La versión de las reglas y el resultado.

La siguiente ejecución, abajo
Cada corrida agrega una línea.

No es una lista [ ]: cada línea es un JSON completo que se lee sola. Por eso se puede agregar sin reescribir el archivo.

ANATOMÍA · ARCHIVO JSON (LINEAGE_TRANSFORMED_DF1.JSON)

```
{
  "recorrido": ["Kaggle (CSV)", "raw_df1 (SQLite)",
    "extracted_df1 (SQLite)", "transformed_df1 (Parquet)"],

  "filas_origen": 16598, "filas_destino": 16323,
  "estrategia_carga": "sobrescribir (full refresh)",
  "version_reglas": "v1",

  "transformaciones": [
    {"paso": "3.3b", "descripcion": "Eliminar filas sin year",
     "filas_antes": 16598, "filas_despues": 16327, …},
    … ],

  "columnas_destino": {"year": "int64",
    "ventas_consistentes": "bool", …}
}
```

[ ] lista
El recorrido completo del dato, en orden.

"clave": valor
Origen, destino, filas y versión de las reglas.

La bitácora del Paso 3
Una lista de objetos: uno por transformación.

{ } dentro de { }
Un diccionario anidado: columna → tipo.

Un solo documento con sangría (indent=2) para que una persona lo lea. Se reescribe completo en cada ejecución.

transformacion_log.jsonl — EL HISTORIAL

```
with open(LOG_PATH, 'a') as f:
    f.write(json.dumps(evento) + '\n')
```

```
{"timestamp": "…T05:50", "filas_salida": 16323}
{"timestamp": "…T06:02", "filas_salida": 16323}
{"timestamp": "…T06:15", "filas_salida": 16323}
```

Crece: una línea por ejecución
'a' = agregar al final. Responde: ¿qué ha pasado?

lineage_transformed_df1.json — LA FOTO ACTUAL

```
with open(LINEAGE_PATH, 'w') as f:
    json.dump(lineage, f, indent=2)
```

```
{
  "recorrido": [ … ],
  "transformaciones": [ … ]
}
```

Se reemplaza: un solo documento
'w' = escribir desde cero. Responde: ¿de dónde vienen estos datos?

ANATOMÍA · ARCHIVOS CSV DE EVIDENCIAS (REPORTES/)

```
# bitacora_transformed_df1.csv
paso,descripcion,filas_antes,filas_despues,filas_afectadas

3.3b,Eliminar filas sin year,16598,16327,271

3.2,"Quitar espacios extra en ['name', 'platform',
    'genre', 'publisher']",16598,16598,5

# calidad_transformed_df1.csv
metrica,antes,después,cambio
filas,16598,16323,-275
celdas_nulas,329,0,-329
```

Encabezado
Las columnas de df_bitacora.

Sin índice: index=False
El 0, 1, 2… no aporta nada.

Comillas si el texto lleva comas
Así no se confunden con separadores.

index_label='metrica'
Aquí el índice SÍ es información: se guarda con nombre.

encoding='utf-8-sig' hace que Excel muestre bien los acentos y la flecha →. Son las tablas de tu reporte: no se copian a mano.

ANATOMÍA · ARCHIVO PARQUET (TRANSFORMED_DF1.PARQUET)

PAR1

Grupo de filas 1

name	year	fabricante	global_sales
Wii Sports	2006	Nintendo	82.74
Super Mario	1985	Nintendo	40.24

Grupo de filas 2 …

Metadatos (footer)

year: INT64 · ventas_consistentes: BOOLEAN

_extracted_at: TIMESTAMP · min / max por columna

PAR1

Datos guardados POR COLUMNA

Cada columna va junta y comprimida.

Esquema y tipos en el footer

Lo que hicimos en 3.6 no se pierde: year regresa como entero.

16,323 filas · 20 columnas

0.75 MB con tu base.

Binario: no se abre en un editor de texto. Se lee con pd.read_parquet() o con DuckDB, que además puede leer solo las columnas que pide.

PASO 1 — Leer el origen: extracted_df1

DIAGRAMA · DOS GRUPOS DE COLUMNAS

COLS_NEGOCIO · 11 columnas → se limpian

Name | Platform | Year | Publisher | Global_Sales | …

COLS_META · 4 columnas → no se tocan

_batch_id | _row_hash | …

```
# Leer en modo solo lectura
conn = sqlite3.connect(f'file:{DB_PATH}?mode=ro', uri=True)

# Separar por el prefijo _
COLS_META    = [c for c in df_ext.columns if c.startswith('_')]
COLS_NEGOCIO = [c for c in df_ext.columns if not c.startswith('_')]
```

_row_hash viaja sin cambios hasta el Parquet: es el lineage de cada fila.

PASO 2 — Diagnóstico de calidad (antes)

No se limpia lo que no se ha medido.

El Paso 2 mide los problemas. El Paso 4 repite la misma medición sobre los datos limpios. La comparación es la evidencia de tu reporte.

LAS CUATRO DIMENSIONES DE CALIDAD · RESULTADO EN VGSALES

Completitud

¿Faltan datos?

`isna().sum()`

329

celdas nulas: Year 271 · Publisher 58

Unicidad

¿Hay filas repetidas?

`duplicated()`

0

duplicados exactos

Validez

¿Los valores son posibles?

`between() · < 0`

4

años fuera de 1980–2016 · 0 ventas negativas

Consistencia

¿Los datos cuadran entre sí?

`global ≈ Σ regiones`

0

inconsistentes · diferencia máxima 0.02

Las preguntas sirven para cualquier dataset. Las **reglas** (qué año es válido, qué columnas deben cuadrar) las define cada dataset.

ANATOMÍA · UNA REVISIÓN DE VALIDEZ, PIEZA POR PIEZA

```
df_ext['Year'].between(ANIO_MIN, ANIO_MAX)

~df_ext['Year'].between(ANIO_MIN, ANIO_MAX)

& df_ext['Year'].notna()

anio_invalido.sum()    # 4

df_ext.loc[anio_invalido, COLS_NEGOCIO]
```

Máscara
Un True / False por fila: ¿está en el rango?

~ invierte
Ahora True significa fuera de rango.

& y además no es nulo
Los nulos ya se contaron en completitud.

Contar los True
True vale 1 y False vale 0.

Ver las filas marcadas
Años 2017 y 2020 en un dataset de 2016.

PASO 3 — Transformaciones

NUEVE TRANSFORMACIONES, CADA UNA LIGADA A UN PROBLEMA

#	Transformación	Tipo	Qué resuelve
3.1	Columnas en snake_case	Normalización	Convención de nombres
3.2	Limpiar texto	Normalización	Espacios que separan iguales
3.3	Tratar nulos	Limpieza	Completitud
3.4	Quitar duplicados exactos	Limpieza	Unicidad
3.5	Rangos y consistencia	Limpieza	Validez y consistencia
3.6	Convertir tipos de dato	Tipado	Año guardado como decimal
3.7	Enriquecer con un catálogo (join)	Enriquecimiento	Agrega el fabricante
3.8	Columnas derivadas	Enriquecimiento	Agrega la década
3.9	Metadatos de transformación	Lineage	Cuándo y con qué versión

ANATOMÍA · LA BITÁCORA DE TRANSFORMACIONES

```
bitacora = []

def registrar(paso, descripcion, filas_antes,
              filas_despues, afectadas):
    bitacora.append({'paso': paso, 'descripcion': ..., ...})

antes = len(df)
df = df.dropna(subset=['year'])

registrar('3.3b', 'Eliminar filas sin year',
          antes, len(df), antes - len(df))
```

Lista vacía
Se reinicia al ejecutar esta celda.

Un registro por paso
append agrega un diccionario a la lista.

Contar, luego transformar
antes guarda las filas previas.

Anotar la diferencia
16,598 − 16,327 = 271 filas.

Ejecutar dos veces una celda de transformación la registra dos veces. Para una bitácora limpia: Ejecutar todas, o desde bitacora = [].

ANATOMÍA · 3.1 A_SNAKE_CASE(): UNA CADENA DE .STR

`pd.Index(columnas)`	`[' Global Sales', 'NA_Sales']`	La lista se vuelve Index: ya tiene .str
`.str.strip()`	`['Global Sales', 'NA_Sales']`	Quita espacios a los lados
`.str.lower()`	`['global sales', 'na_sales']`	Todo a minúsculas
`.str.replace(' ', '_')`	`['global_sales', 'na_sales']`	Espacios internos → _
`list(...)`	`['global_sales', 'na_sales']`	De vuelta a lista normal

```
df.columns  = a_snake_case(df.columns)
COLS_TEXTO  = a_snake_case(COLS_TEXTO)   # las reglas también cambian de nombre
```

Una lista de Python no tiene .str. Sin pd.Index habría que recorrer cada nombre con un ciclo for.

3.3 TRATAR NULOS · CADA COLUMNA SE DECIDE POR SU SIGNIFICADO

`publisher` · `58 nulos`

Imputar 'Unknown'

El juego existe y sus ventas son válidas; solo falta la editorial.

```
df['publisher'].fillna('Unknown')
```

`year` · `271 nulos`

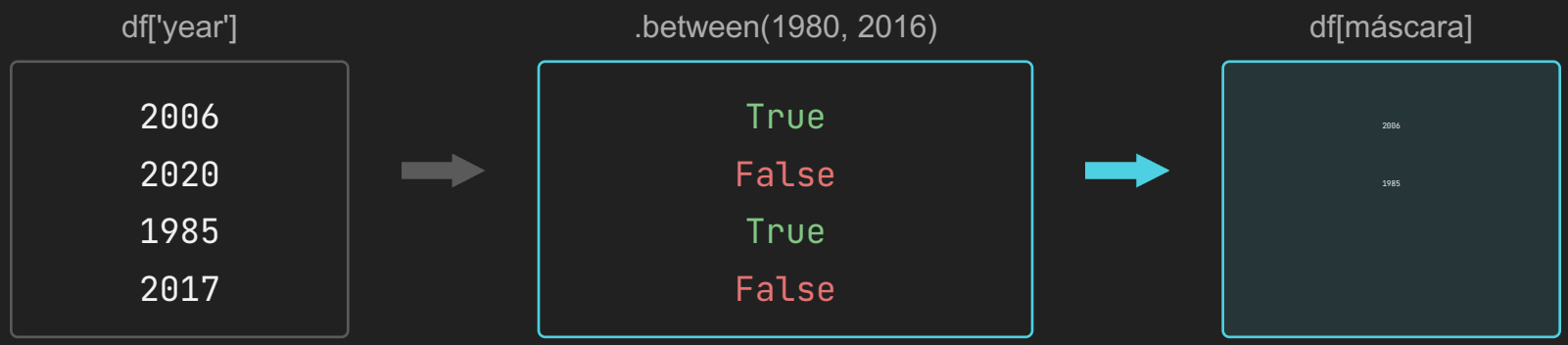
Eliminar la fila

Inventar un año sería un dato falso, y sin año no sirve para tendencias. Son 1.6% de las filas.

```
df.dropna(subset=['year'])
```

En tu dataset las decisiones pueden ser otras. Lo que se evalúa es la **justificación** en el reporte.

DIAGRAMA · ELIMINAR = QUEDARSE CON LO VÁLIDO



Una columna → un True/False por fila

```
df = df[df['year'].between(ANIO_MIN, ANIO_MAX)]
```

Varias columnas → resumir con .all(axis=1)

```
df = df[(df[COLS_VENTAS] >= 0).all(axis=1)]
```

Las filas desaparecen en el **df =**: el nuevo df ya no las tiene. Sin él, el filtro solo se vería en pantalla.

ELIMINAR · validez

```
# year fuera de [1980, 2016]
df = df[df['year'].between(ANIO_MIN, ANIO_MAX)]

# ventas negativas
df = df[(df[COLS_VENTAS] >= 0).all(axis=1)]
```

El valor es imposible
Un error de captura: la fila no sirve. 4 filas eliminadas.

MARCAR · consistencia

```
diferencia = (df[COL_GLOBAL]
    - df[COLS_REGIONES].sum(axis=1)).abs()

df['ventas_consistentes'] =
    diferencia <= TOLERANCIA
```

La fila puede ser real
Nueva columna True/False: quien analiza decide. .abs(): |−0.10| = 0.10.

3.6 CONVERTIR TIPOS DE DATO

columna	antes	después	por qué
year	float64	int64	tenía nulos; ya no
_extracted_at	object	datetime64[us]	SQLite la guardó como texto
ventas (×5)	float64	float64	redondeo a 2 decimales
ventas_consistentes	—	bool	columna nueva de 3.5

```
df['year']          = df['year'].astype('int64')
df['_extracted_at'] = pd.to_datetime(df['_extracted_at']).astype('datetime64[us]')
```

Parquet guardará estos tipos dentro del archivo: en el Paso 5 los comprobamos.

Tipos de pandas: pandas.pydata.org/docs/reference/arrays.html · en pandas 3 el texto aparece como str en lugar de object

DIAGRAMA · 3.7 ENRIQUECER CON UN CATÁLOGO (JOIN)

df · 16,323 filas

name	platform
Wii Sports	Wii
Gran Tur…	PS
Halo 3	X360

+

catálogo · 31 filas

platform	fabricante
Wii	Nintendo
PS	Sony
X360	Microsoft

→

df · 16,323 filas · +1 columna

name	platform	fabricante
Wii Sports	Wii	Nintendo
Gran Tur…	PS	Sony
Halo 3	X360	Microsoft

how='left'

Conserva todas las filas de df, aunque no tengan pareja.

validate='m:1'

Error si el catálogo repite una plataforma: no multiplica filas.

indicator=True

Marca las filas sin pareja; reciben 'Otro'. Con tu base: 0.

LA BITÁCORA CON TU BASE · 12 REGISTROS

16,323

filas limpias, de 16,598

275 filas eliminadas

paso	descripción	afectadas
3.2	Quitar espacios extra en texto	5 celdas
3.3a	publisher nulo → 'Unknown'	58 celdas
3.3b	Eliminar filas sin year	271 filas
3.5a	Eliminar year fuera de [1980, 2016]	4 filas
…	8 pasos más sin filas afectadas	0

Un paso con 0 también se registra: demuestra que la regla **se revisó** y protege las siguientes ejecuciones.

PASO 4 — Diagnóstico de calidad (después)

LAS MISMAS MEDICIONES DEL PASO 2, AHORA SOBRE DF_FINAL

métrica	antes	después	cambio
filas	16598	16323	-275
celdas_nulas	329	0	-329
duplicados_exactos	0	0	0
anios_fuera_de_rango	4	0	-4
ventas_negativas	0	0	0
ventas_inconsistentes	0	0	0

Esta tabla es la evidencia central del reporte: qué problemas había y qué quedó.

¿Cuadra la bitácora?

```
bitácora : 275
real     : 275
Cuadra
```

ventas_inconsistentes puede quedar arriba de 0: esas filas se **marcaron**, no se eliminaron.

PASO 5 — Carga: escribir en Parquet

La "L" de ETL

¿POR QUÉ PARQUET Y NO EL .DB?

Hasta aquí, df_final solo vive en memoria: si cierras Colab, se pierde. La carga lo deja en un destino persistente.

Así trabaja la industria

La capa limpia (silver) vive en archivos Parquet dentro de un data lake. La base de datos llega al modelar, en la Unidad 3.

Conserva los tipos

Lo que hicimos en 3.6 queda dentro del archivo. SQLite y CSV no lo conservan.

Se consulta con SQL

DuckDB lee el Parquet directamente, sin cargarlo a ninguna base (Paso 7).

EXTRACTED · APPEND

```
nuevas.to_sql('extracted_df1', conn,
              if_exists='append')

# agrega el lote nuevo al final
```

Cada lote es historia
Si se borra, no se puede recuperar.

TRANSFORMED · SOBRESCRIBIR

```
df_final.to_parquet(PARQUET_DESTINO,
                    index=False)

# si el archivo existe, lo reemplaza
```

Se recalcula entero
Siempre se puede volver a generar desde EXTRACTED.

Idempotencia: ejecutar el pipeline una o diez veces deja el mismo resultado. Si cambia una regla, basta con volver a correr el notebook.

ANATOMÍA · LA CELDA DE CARGA

```
df_final.to_parquet(PARQUET_DESTINO, index=False)


tamanio_mb = os.path.getsize(PARQUET_DESTINO) / 1_048_576

escribir_log('carga_parquet', len(df_ext), len(df_final),
             detalle=f'{tamanio_mb:.2f} MB')


df_leido = pd.read_parquet(PARQUET_DESTINO)

# extracted_df1 → transformed_df1.parquet
#   Filas de entrada : 16,598
#   Filas cargadas   : 16,323
#   Tamaño           : 0.75 MB
```

Escribir el archivo
En Drive; si ya existe, se sobrescribe.

Tamaño en MB
1 MB = 1,048,576 bytes.

Una línea más en el log
El historial de la operación.

Leerlo de vuelta
Mismos tipos que en memoria.

Resultado con tu base
Lo que imprime la celda.

PASO 6 — Lineage y evidencias

DIAGRAMA · EL LINAJE EN DOS NIVELES



De la TABLA

`lineage_transformed_df1.json`

Un archivo por tabla. ¿De dónde salió? ¿Qué reglas se aplicaron, en qué orden y cuántas filas quitó cada una?

De cada FILA

`_row_hash · _batch_id · _extracted_at`

Columnas dentro del Parquet. ¿De qué fila exacta de extracted_df1 y raw_df1 viene? ¿En qué lote llegó?

Lo común a todas las filas se escribe **una vez**; lo que cambia por fila viaja **en sus columnas**.

PASO 7 — Validación con DuckDB

Sobre el archivo ya cargado

ANATOMÍA · SQL DIRECTO SOBRE UN PARQUET

```
import duckdb


con = duckdb.connect()


P = f"read_parquet('{PARQUET_DESTINO}')"

con.sql(f'''
    SELECT name, platform, fabricante, year, global_sales
    FROM {P}
    ORDER BY global_sales DESC LIMIT 5
''').df()

con.close()
```

Una librería de Python
En Colab suele venir instalada.

Base en memoria
No crea ningún archivo.

El archivo como tabla
Así se lee un Parquet dentro de SQL.

El SQL de siempre
.df() regresa un DataFrame de pandas.

Cerrar al terminar
Igual que con SQLite.

Si no viene instalado: !pip install duckdb. Es el puente hacia la Unidad 3.

DOS PRUEBAS SOBRE EL ARCHIVO CARGADO

ESQUEMA · ¿qué tipos quedaron?

```
con.sql(f'DESCRIBE SELECT * FROM {P}').df()
```

column_name	column_type
year	BIGINT
name	VARCHAR
global_sales	DOUBLE
ventas_consistentes	BOOLEAN
_extracted_at	TIMESTAMP

CONSTRAINTS · ¿filas completas?

```
SELECT COUNT(*) AS filas,
  SUM(CASE WHEN year IS NULL
           THEN 1 ELSE 0 END) AS nulos_year,
  SUM(CASE WHEN publisher IS NULL
           THEN 1 ELSE 0 END) AS nulos_publisher
FROM {P}
```

filas	nulos_year	nulos_publisher
16323	0	0

Se valida **lo que quedó en el destino**, no lo que había en memoria. Esto completa el reporte de validación del Hito 2.

ERRORES FRECUENTES

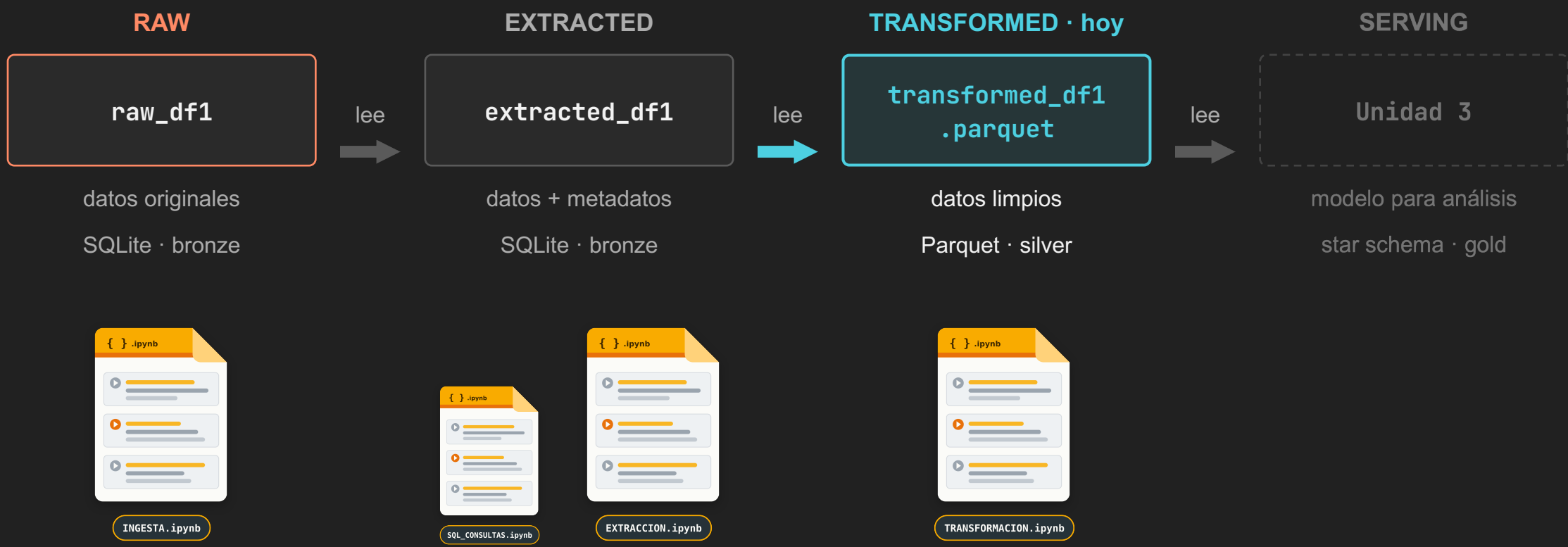
- ⚠ Ejecutar dos veces una celda de transformación: la bitácora registra ese paso dos veces.
- ⚠ Renombrar las columnas sin actualizar las listas de reglas (COLS_TEXTO, COLS_VENTAS): KeyError más adelante.
- ⚠ Comparar decimales con == en lugar de una tolerancia: cientos de filas “inconsistentes” que están bien.
- ⚠ Usar between() sin notna(): los nulos se cuentan también como fuera de rango.
- ⚠ Creer que df_final ya está guardado: sin el Paso 5 se pierde al cerrar Colab.
- ⚠ Leer raw_df1 en lugar de extracted_df1: rompe la regla de oro.

LA CAPA TRANSFORMED

medir · limpiar · registrar
cargar · validar

Siguiente: en la Unidad 3, los Parquet de Processed/ son la fuente del star schema.

DIAGRAMA · ¿EN QUÉ PARTE DEL PIPELINE ESTAMOS?



Regla de oro: cada capa lee solo de la anterior. Este notebook lee extracted_df1 en modo solo lectura, nunca raw_df1.

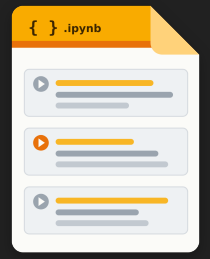
DIAGRAMA · ¿EN QUÉ PARTE DEL PIPELINE ESTAMOS?

RAW

`raw_df1`

datos originales

SQLite · bronze



INGESTA.ipynb

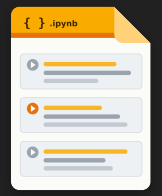
lee

EXTRACTED

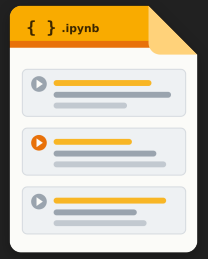
`extracted_df1`

datos + metadatos

SQLite · bronze



SQL_CONSULTAS.ipynb



EXTRACCION.ipynb

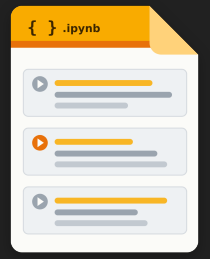
lee

TRANSFORMED · hoy

`transformed_df1 .parquet`

datos limpios

Parquet · silver



TRANSFORMACION.ipynb

lee

SERVING

`Unidad 3`

modelo para análisis

star schema · gold

ACTIVIDAD 2